Culpa

When an AI agent buys the wrong thing, who pays?

Culpa is the referee for agentic commerce. It decides whether the buyer, the merchant or the agent provider caused a problem, and settles the dispute on PayPal with the evidence to back it up.

Product flow walkthrough · PayPal AI Hackathon 2026

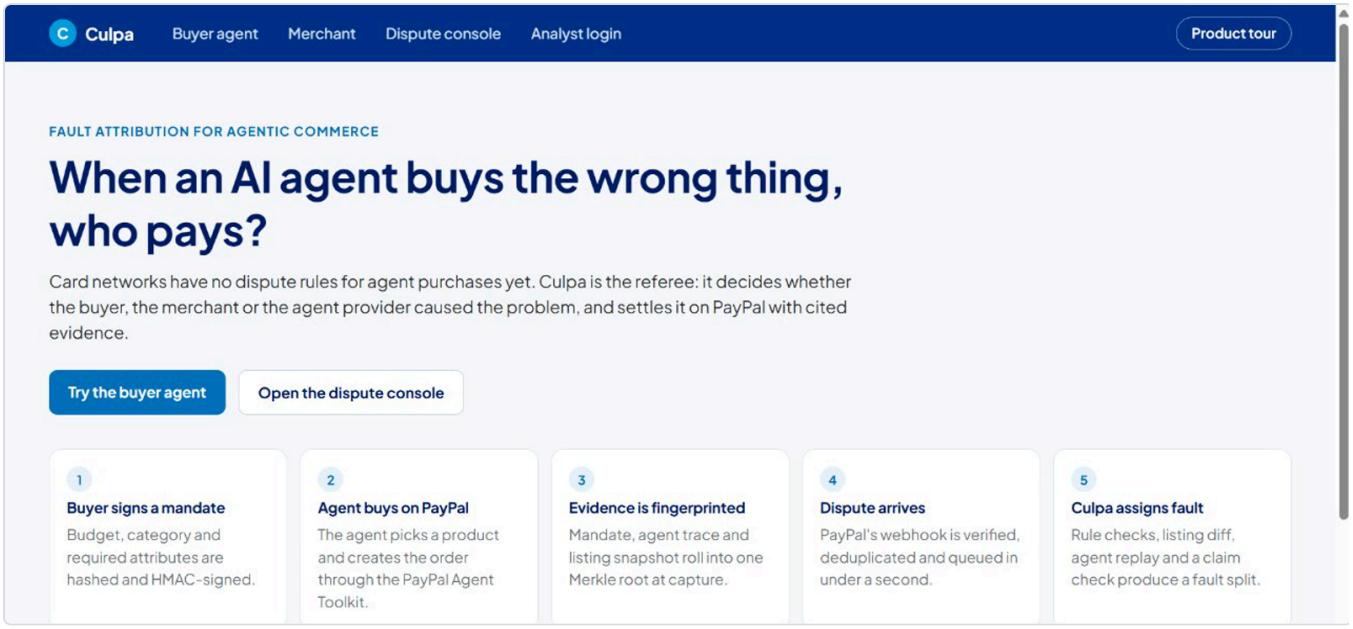
The problem

AI agents now shop for people. When an agent buys the wrong item, today's dispute system cannot tell whether the instruction was unclear, the agent ignored it, the merchant shipped something different, or the buyer is blaming the agent to get a refund. Card networks still have no dispute category for agent purchases, so PayPal and merchants absorb the losses. Most agent safety tools stop a bad payment before it happens; nothing decides fault after the money has moved.

The flow at a glance

1 Mandate | 2 Agent search | 3 PayPal approval | 4 Fulfilment | 5 Console | 6 Dispute | 7 Verdict | 8 Evidence | 9 Settlement | 10 Outcome

Three roles take part: the buyer, who instructs the agent and pays; the merchant, who ships; and the analyst, who settles disputes in the console. The screenshots in this document come from a recorded run of the product against the PayPal sandbox.

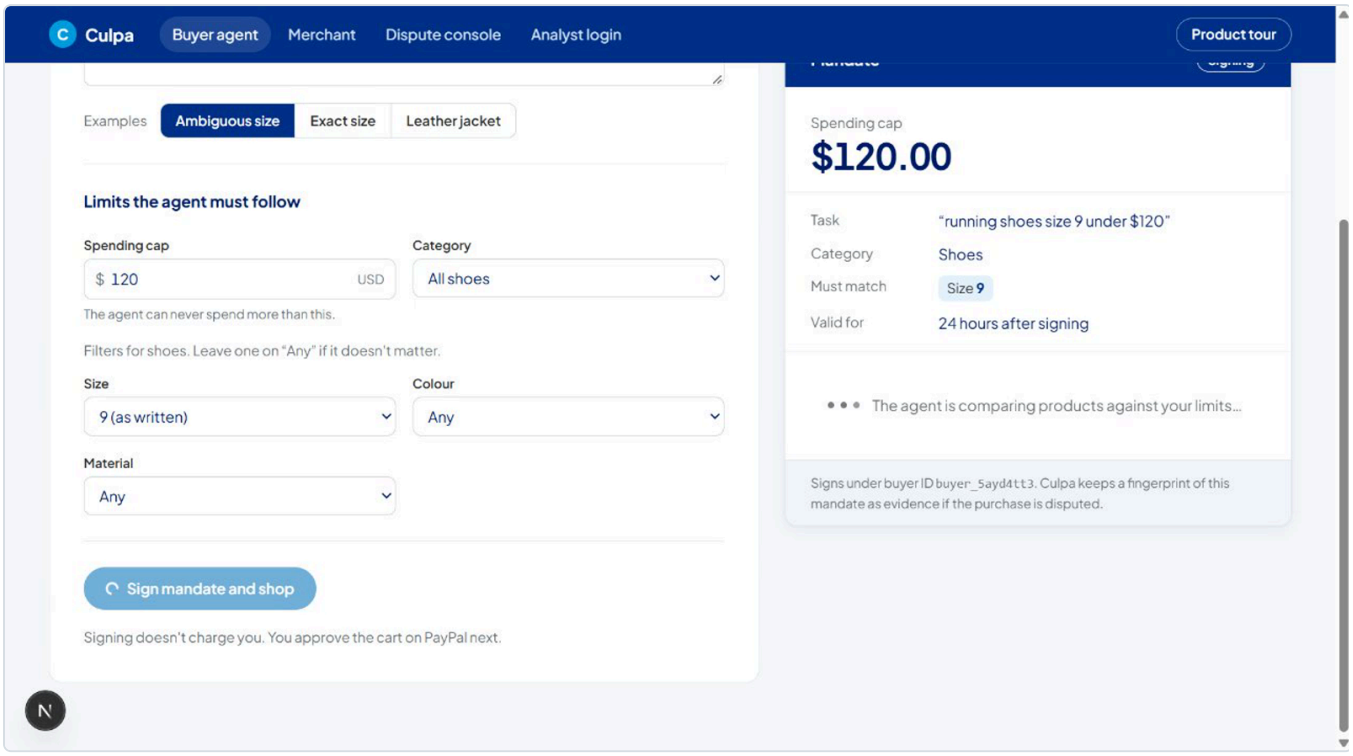


1 The buyer writes a mandate

Buyer agent page

The buyer tells the agent what to buy in plain words, sets a spending cap and picks the details that must match. The form detects the category from the request and offers only the filters that exist for it, so earphones never ask for a shoe size.

- Signing turns these instructions into a mandate that cannot be changed later.
- The mandate panel on the right previews exactly what will be signed.

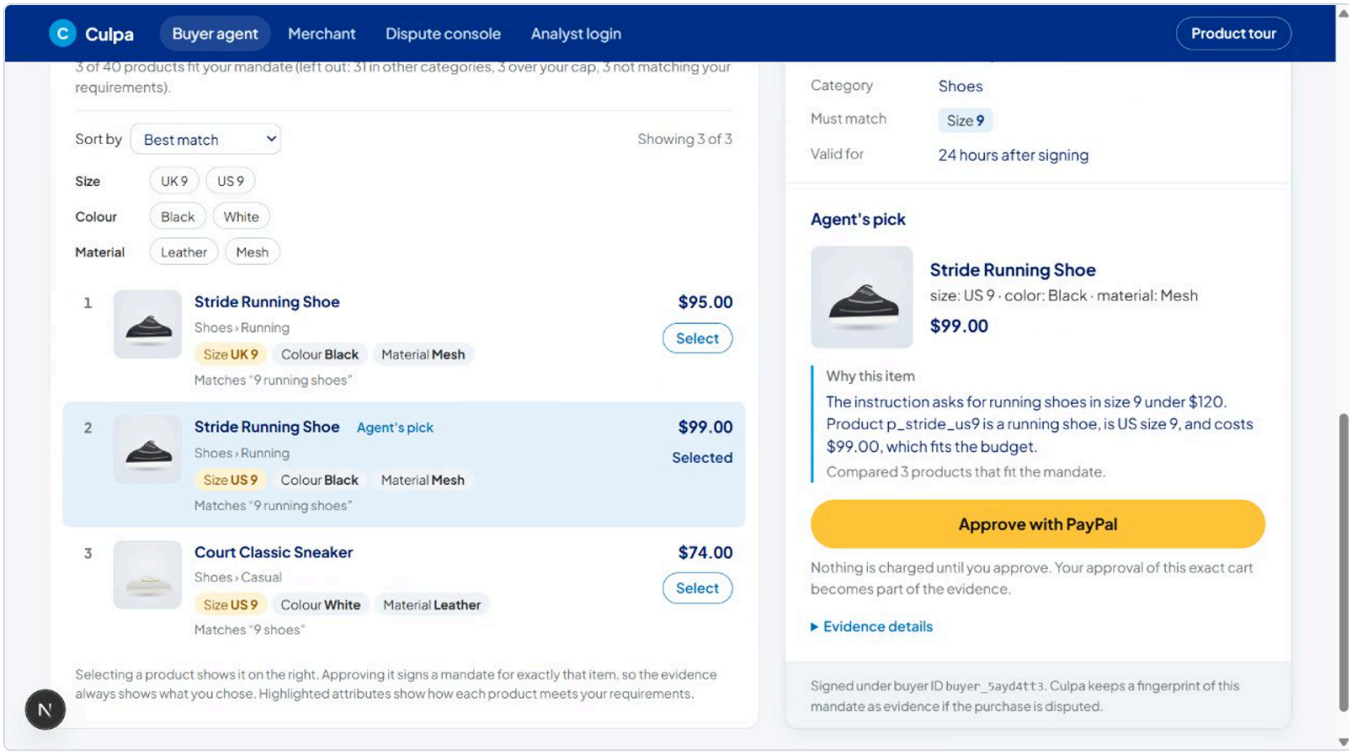


2 The agent searches the store

Buyer agent page, search results

The agent leaves out products in other categories, over the cap or not matching a required attribute, then ranks what is left by how closely each one matches the request.

- The buyer sees every match with filters and sorting, not a single hidden choice.
- The agent's pick is highlighted; the buyer can keep it or select another product.
- Nothing is charged yet: the next step is approval on PayPal.

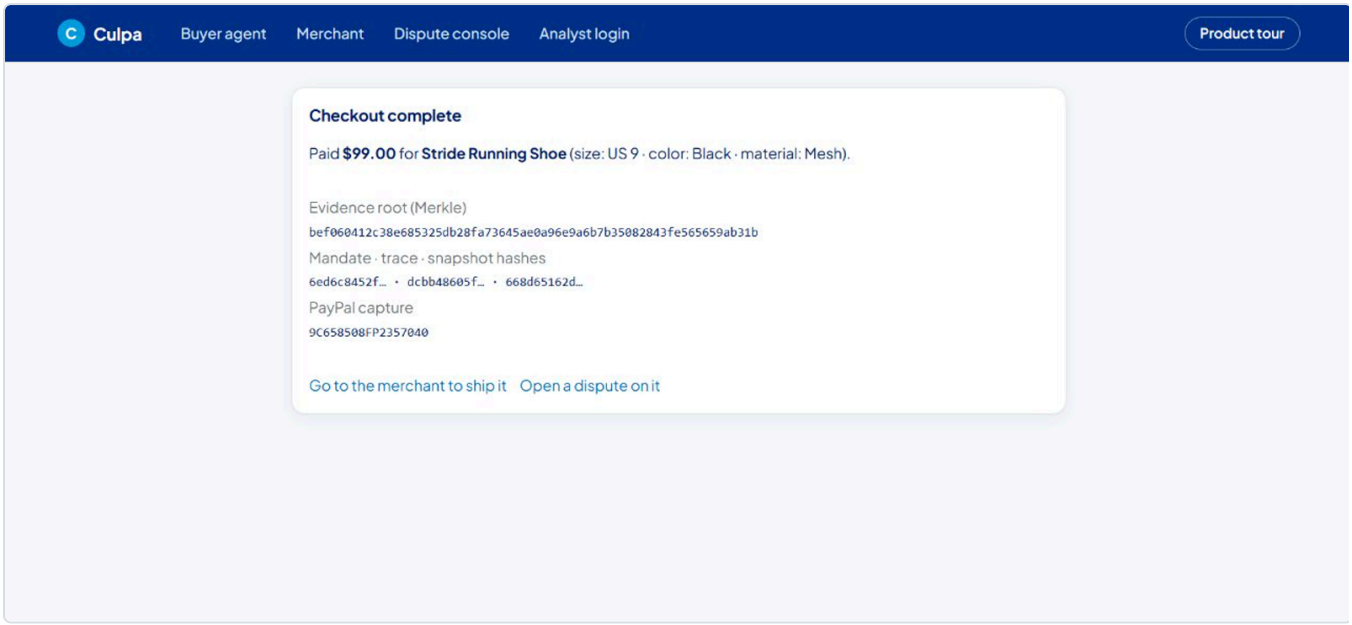


3 The buyer approves on PayPal

PayPal sandbox, then the checkout page

The order is created through the PayPal Agent Toolkit. The buyer logs in to PayPal and approves the exact cart. When the payment is captured, Culpa fingerprints the evidence.

- The mandate, the agent's decision record and the listing as it looked at purchase are each hashed.
- The hashes form one tamper-evident root, stored with the order and linked to the PayPal order.
- If anyone edits the evidence later, the root no longer matches and the case goes to a person.



4 The merchant fulfils the order

Merchant page

The merchant sees orders waiting to ship. Shipping sends whatever the listing says at that moment, so a merchant who changes a listing after the sale ships something different from what was bought. Culpa keeps every version of every listing to prove it.

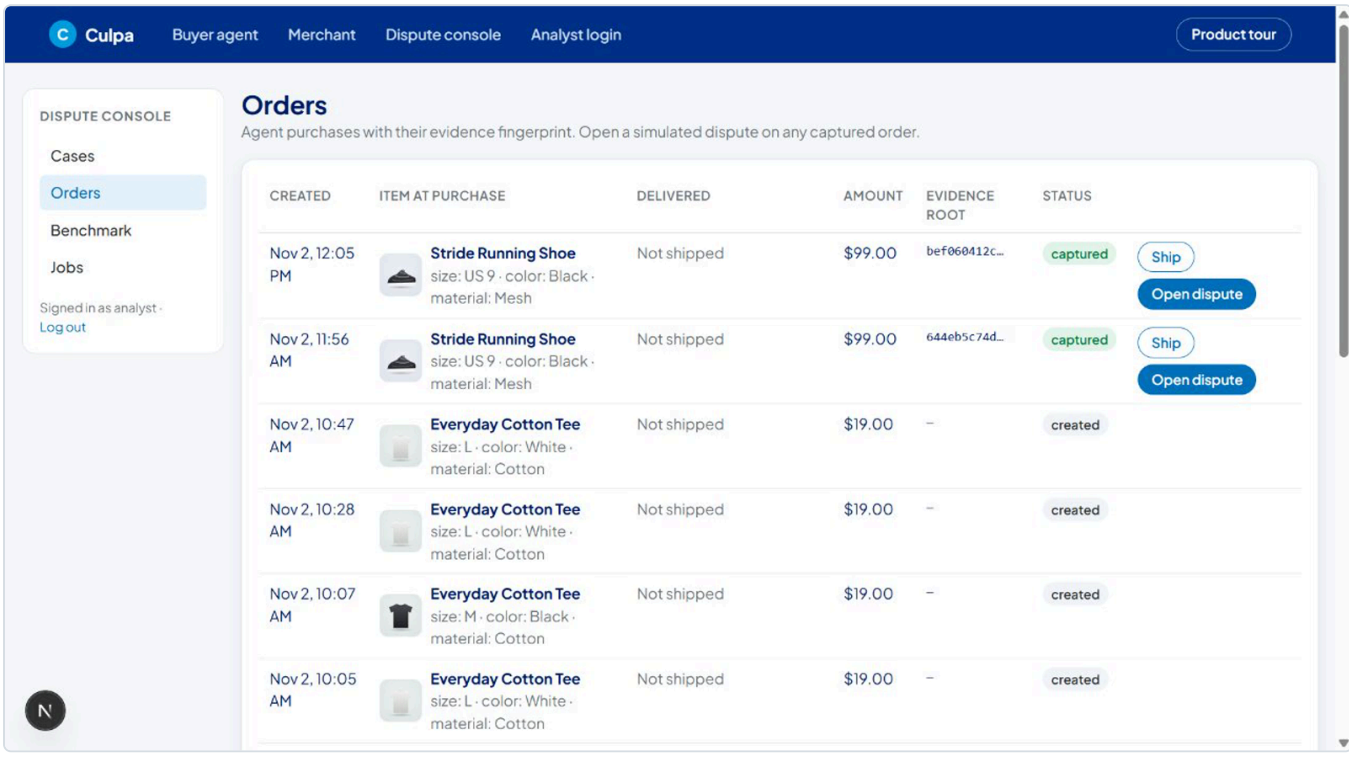
- In the recorded demo the order was not shipped, which decides the verdict in step 7.

5 The analyst opens the dispute console

Dispute console, Orders

Disputes are handled by an analyst who logs in to the console. The Orders page lists every agent purchase with its evidence fingerprint.

- Real disputes arrive from PayPal webhooks or a scheduled check of PayPal's dispute list.
- For demos, the same path can be started from the built-in dispute simulator.

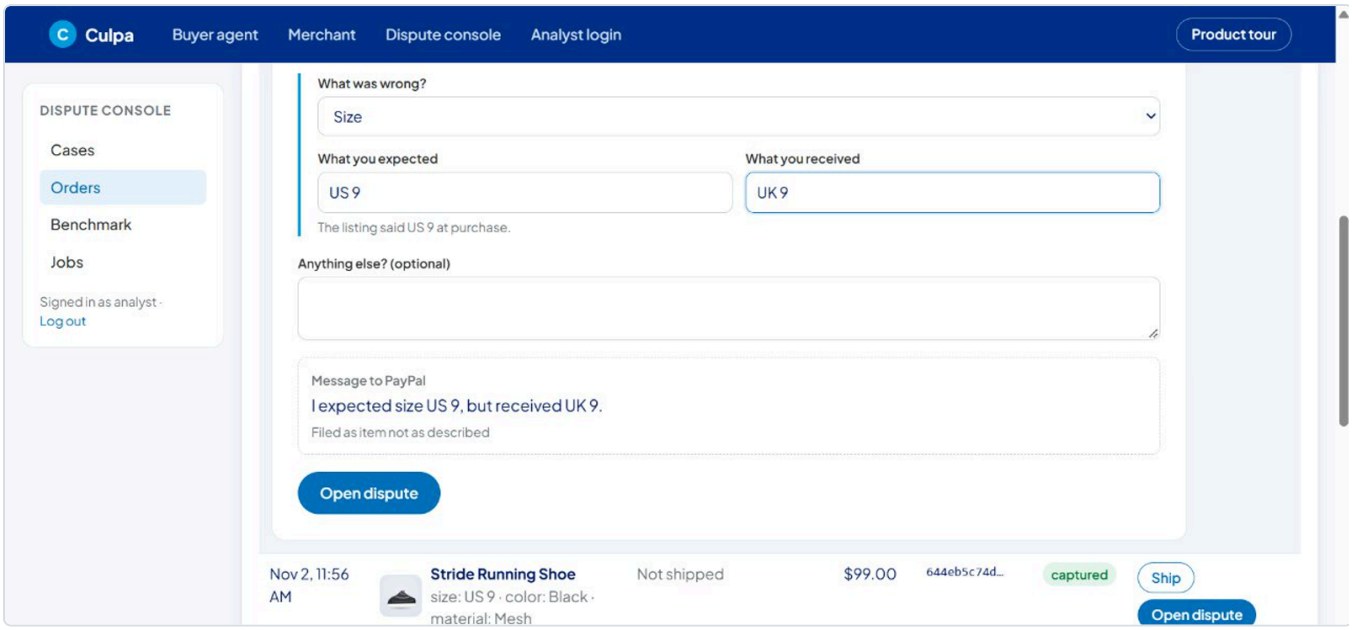


6 A dispute is filed

Open dispute form

The dispute form asks what went wrong in plain language: not received, not as described, the agent bought the wrong thing, not authorised, or something else. For “not as described” it asks which detail was wrong and what was expected and received.

- The form writes the buyer's message to PayPal and files it under the matching PayPal reason.
- In the demo: expected size US 9, received UK 9.

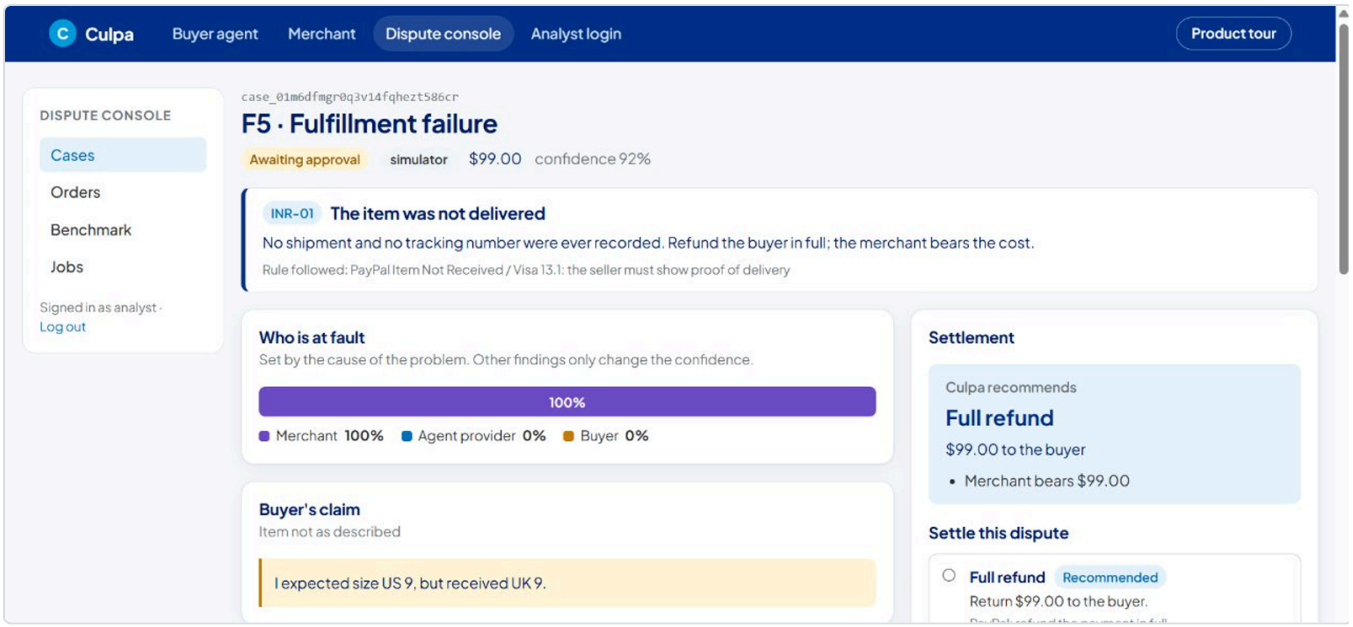


7 Culpa decides who is at fault

Case page, verdict

Culpa runs four checks in order: the mandate's hard limits, the delivery against the listing at purchase, a replay of the agent's decision, and the buyer's claim against the record, with a review by Google Gemini.

- Fault follows the cause, not a vote: the party that caused the problem is 100% at fault.
- Every verdict has a plain reason and the real rule it follows.
- In the demo: nothing was ever shipped, so the merchant is at fault (INR-01, PayPal Item Not Received / Visa 13.1) and a full refund is recommended.

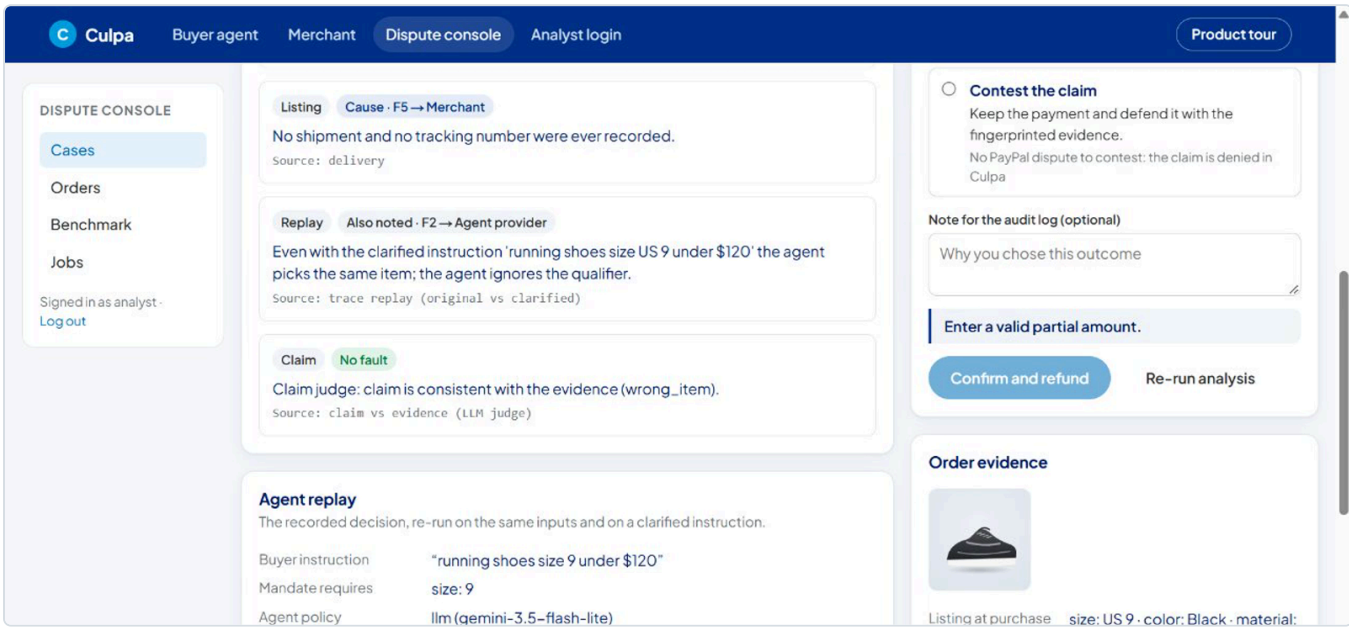


8 The evidence behind the verdict

Case page, evidence

Each finding is labelled as the cause, an issue also noted, or a supporting signal. Supporting signals, such as dispute history or the AI review, only change the confidence, never the share of blame.

- The agent replay shows the original decision and a re-run on a clarified instruction.
- A timeline and a hash-chained audit trail record every step.

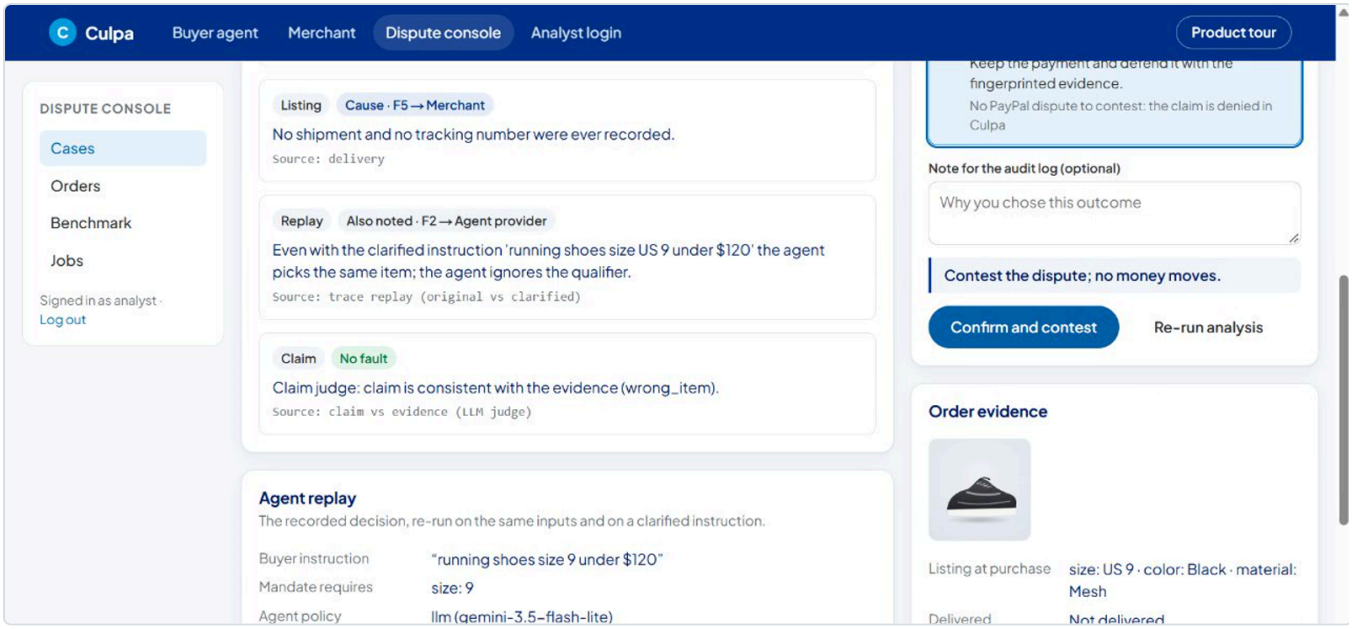


9 The analyst settles the dispute

Case page, settlement

Culpa recommends an action and shows who bears the cost. The analyst chooses a full refund, a partial refund of an amount they set, or to contest the claim, and can add a note for the audit log.

- Each option states exactly what happens on PayPal: accept the claim, send a refund offer, refund the payment or submit Culpa's evidence.
- Code moves the money, never the AI, and every PayPal request carries an idempotency key so it can never happen twice.

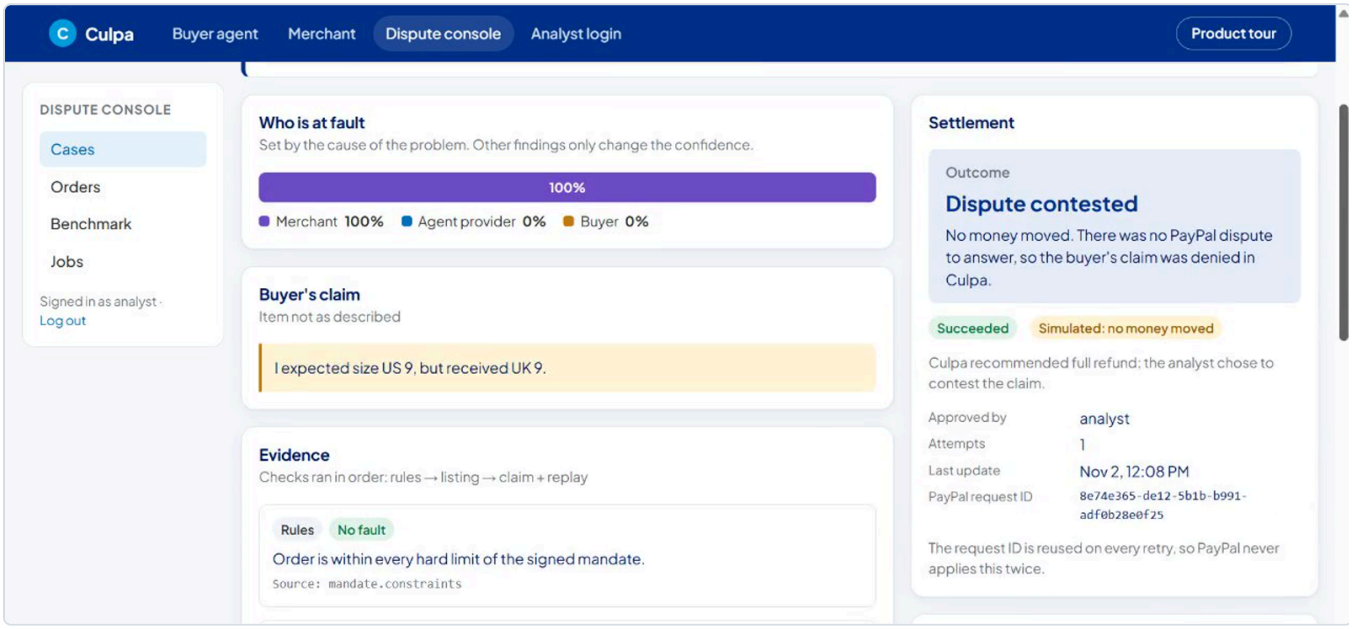


10 The outcome is recorded

Case page, outcome

After settling, the panel shows what actually happened in plain words. In the demo the analyst chose to contest, so it reads “Dispute contested. No money moved.” and notes that this differs from Culpa's recommendation.

- The decision, who made it and the PayPal request ID are kept in the audit log.



How fault is decided

Fault follows the cause of the problem. When several causes exist, the one the buyer's loss actually comes from wins, in the order below.

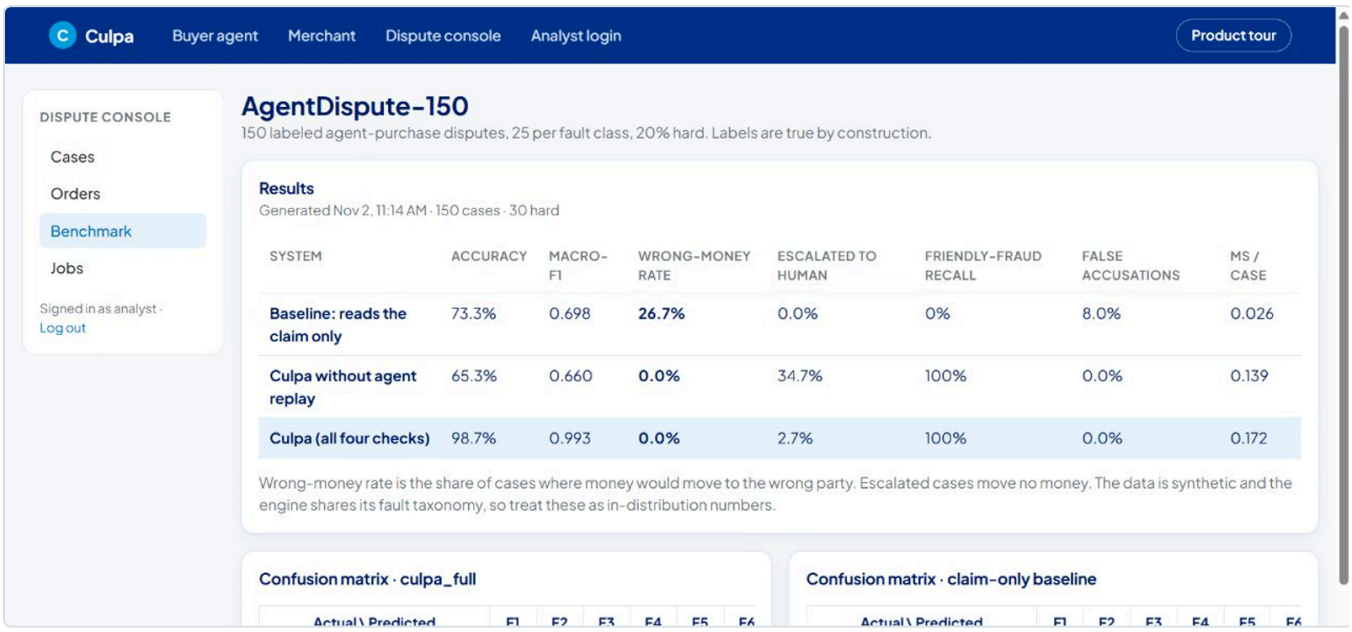
What the evidence shows	Who is at fault
Nothing delivered	Merchant
Delivered item differs from the listing	Merchant
Agent broke a hard limit of the mandate	Agent provider
Agent chose the wrong item	Agent provider
Instruction was genuinely ambiguous	Buyer and agent provider (shared)
Claim contradicts the recorded evidence	Buyer
Damaged, defective, counterfeit, duplicate charge or missing refund	No percentage: sent to a person with the evidence needed

Why it can be trusted with money

- The AI model never moves money. Gemini receives the buyer's text only as quoted data, answers in a fixed format, has no tools, and only adjusts confidence.
- A settlement policy written in code decides what happens to money; anything below 80% confidence waits for a person.
- Every PayPal request carries an idempotency key, settlements follow a strict state machine, and webhooks are signature-checked and deduplicated.
- Every verdict, approval and PayPal action goes into an append-only audit log where each entry contains the hash of the one before it.
- Culpa never sees card data, because PayPal hosts payment entry.

Results

AgentDispute-150 is a set of 150 synthetic disputes, 25 per fault type, a fifth of them deliberately hard. A baseline that reads only the buyer's claim is right 73.3% of the time and sends money to the wrong party in 26.7% of cases. Culpa is right 98.7% of the time, sends money to the wrong party in 0% of cases and passes 2.7% to a person. A second set of 126 scenarios built from PayPal Buyer Protection rules and Visa and Mastercard reason codes passes in full. The data is synthetic, so these are in-distribution numbers, not real-world accuracy.



Running it

Requirements: Python 3.11 or newer, Node 20 or newer and pnpm. From the repository root on Windows:

- First-time setup: `powershell -ExecutionPolicy Bypass -File scripts\setup.ps1`
- Start the app: `powershell -ExecutionPolicy Bypass -File scripts\dev.ps1`
- Run every check: `powershell -ExecutionPolicy Bypass -File scripts\test.ps1`

The web app opens at http://127.0.0.1:3000 and the API documentation at http://127.0.0.1:8000/docs. Demo logins for the dispute console and the PayPal sandbox buyer are in START_HERE.md, and the full product description is in README.md.